

Kata: an executable quality gate for AI-assisted software changes

Walter Aoiama Nagai ¹¶

¹ Instituto de Ciências Tecnológicas, Universidade Federal de Itajubá, Brazil  ¶ Corresponding author

DOI: [N/A](#)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: 01 January 1970

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Kata (the Japanese word for “form” or “pattern”, as in a martial arts kata: a disciplined, repeatable sequence of movements) is a quality gate for software changes assisted by artificial intelligence (AI) agents. When an AI coding agent says “done”, Kata does not take the claim at face value: it executes the checks the agent claims to have passed, compares the claims against the actual repository state, and refuses to approve work that does not hold up. Kata guides a change through a nine-phase cycle — FIT, THINK, SIMPLIFY, INTENT, SURGICAL, VERIFY, TWIN CHECK, ARTIFACT, REPORT — plus an optional adversarial JUDGE phase that re-runs every claimed check and hunts for seven categories of fraud, such as weakened tests, false completion claims, and undeclared scope. Every phase is recorded in a task file (`.kata/<task>.yaml`), so the evidence behind an approval is persistent and auditable, not conversational. Kata is implemented as a Python command-line interface (CLI) with two agent frontends: an OpenCode agent and a set of Claude Code skills, both generated from a single source of truth. It is designed for developers, researchers, and teams who use AI agents to modify code and need objective, reproducible verification that what was claimed was actually done.

Statement of need

AI coding agents produce code faster than any developer can review, and the oversight surface collapses onto the automated test suite. Reward hacking naturally arises in this setup: agents optimize for passing visible tests while deviating from the user’s true goal, and the gap between visible and held-out test suites grows by 28 percentage points for every tenfold increase in code size ([Zhao et al., 2026](#)). Agents also end failure transcripts with “all tests pass”, and tests get weakened until they agree with the code ([Sahir619, 2026](#)). Large language models cannot reliably self-correct without external feedback — sometimes their performance even degrades after self-correction ([Huang et al., 2024](#)). The implication is that verification cannot be left to the model checking itself: it must be external, executable, and objective.

Existing responses to this problem are mostly instructions. Skills and methodologies such as the Fable Method ([Sahir619, 2026](#)) tell the agent what to value (“be careful, verify your work”) and what to do, in order, with thresholds. They improve behavior, but they depend on the model following the instruction: nothing executes the checks, and nothing detects when a phase was filled in without being observed. Kata’s contribution is to move the objective parts of the discipline out of the prompt and into executable, testable Python code. The lint, test, and coverage steps read real exit codes; the JUDGE re-runs claimed checks against the real repository, including committed and untracked files; and the audit mode grades each phase of a completed task as followed, skipped, or faked, naming the concrete risk each skip or fake created. A skill asks for honesty; Kata detects its absence.

State of the field

Several tools and methodologies address disciplined AI-assisted development. The Fable Method ([Sahir619, 2026](#)) is the closest conceptual ancestor: it defines a think/act/prove loop with a fit gate, a triviality gate, evidence before action, adversarial verification, and outcome-first reporting, and it ships an eval that keeps the method honest. Kata adapts the fit gate, the triviality gate, the intent gate, and the hard bounds directly from it, but differs in one structural way: the objective logic lives in a Python package (`kata.fit`, `kata.verify`, `kata.judge`) that is unit-tested and measured for coverage, rather than in prompt text. claude-wizard ([vlad-ko, 2026](#)) is an 8-phase development skill with TDD and adversarial review, but it is a skill only: there is no backend that executes the gates. Ring ([LerianStudio, 2026](#)) is a large skills library (76 skills, 33 agents) that enforces engineering practices with 10-gate development cycles, again as instructions rather than executable checks. nova ([TeamSPWK, 2026](#)) provides pre-commit quality gating, but as an agent skill without a shared executable core. On the research side, SpecBench ([Zhao et al., 2026](#)) measures reward hacking in coding agents, and SWE-bench ([Jimenez et al., 2024](#)) benchmarks agent capability on real GitHub issues; both measure agents, neither provides a gate that a team can run on its own changes.

Kata was built rather than contributed to existing projects for three reasons. First, no existing tool executes the verification: the closest alternatives are prompts that ask the model to verify itself, which the self-correction literature shows is unreliable ([Huang et al., 2024](#)). Second, Kata needed to serve two agent hosts (OpenCode and Claude Code) with identical behavior; a single Python backend with generated frontends guarantees that the checks a task claims to have passed are the same checks in both hosts. Third, Kata needed to be verifiable itself: its own lint, tests, and coverage are measured, and 19 adversarial trap scenarios run in continuous integration to ensure the JUDGE catches planted frauds and does not accuse honest work.

Software design

Kata's design is organized around one principle: the parts of the cycle that can be objective must be executable, and the parts that cannot must be declared before the evidence exists. The Python package `src/kata/` implements the objective logic in four modules. `kata.fit` measures the real diff against HEAD (including untracked files) and applies the triviality gate (at most one changed file and fewer than ten changed lines). `kata.verify` runs lint, tests, and coverage with a numeric gate, reading exit codes rather than model intentions; a declared role in `.kata/config.yaml` runs verbatim, so the tool does not assume the target project is Python. `kata.judge` treats the task file as a collection of claims, re-runs the claimed checks, diffs the claims against the repository, and hunts seven fraud categories: weakened checks, false completion, scope creep, unauthorized actions, specification betrayal, debris, and baseline tampering. `kata.cli` orchestrates the nine phases, persists the task file, and exposes the audit and judge modes.

The cycle itself encodes two design trade-offs worth making explicit. The first is the done criterion (Fable Step 1): THINK records what “ready” means and how it will be verified *before* the evidence exists, and VERIFY confronts that declared criterion with the final result. Without this, the success criterion only exists after the work is done, and “is it satisfied?” becomes unfalsifiable. The second is the hard bound (Fable Step 5): after three failed verification attempts the task is handed back to the user with what was tried, the real output, and the current hypothesis, instead of looping fix-verify forever. The JUDGE also confesses its blind spots: if it could not observe something (a test language it has no probes for, a file outside the diff), the verdict is UNVERIFIABLE, never VERIFIED — “I could not look” is never reported as “all clear”.

The frontends are generated, not hand-maintained. The phase prompts live once in `phases/*.md` and are rendered into the OpenCode agent and the Claude Code skills by a build script; 93%

of the content is shared, and the remaining 7% is declared difference (tool names of the host). This single-source design exists because the previous arrangement — two hand-maintained copies — failed: the copies accumulated 395 divergent lines, including an improvement applied to one frontend and forgotten in the other. A test fails when the generated files drift from the source, so the divergence cannot pass unnoticed.

Research impact statement

Kata is dogfooded: its own lint, tests, and coverage are measured (gate 70%), and the continuous integration runs the Makefile rather than restating the commands, so what is verified locally and remotely cannot drift apart. The adversarial evaluation is the strongest evidence of impact: 19 trap scenarios (`eval/run_traps.py`) plant frauds the JUDGE must catch and honest work it must not accuse. Twelve scenarios plant a fraud that must be detected (weakened checks, false completion, scope creep, unauthorized action, specification betrayal, debris, baseline tampering); four are entirely honest tasks that must come back VERIFIED; two expect UNVERIFIABLE for confessed blind spots. The trap suite includes a guard against refusing legitimate work: one scenario plants real debris beside files whose names merely look like debris, and a judge that refuses honest work is treated as broken as one that misses fraud. The unit suite also exercises the JUDGE against a real git repository in a temporary directory, because blindness to committed or untracked changes cannot be reproduced with mocks. The schema of the task file is compatible with the `.karpathy/` schema of the mushin agent (a private local project where the cycle was first implemented), so legacy tasks migrate through a symbolic link. The repository is public, MIT-licensed, and installable in both agent hosts through symlink installers.

AI usage disclosure

Generative AI tools were used in the development of this software and in the writing of this manuscript. The software itself is a tool for verifying AI-assisted code changes, and its development followed the cycle it implements: the phase prompts were drafted with AI assistance and validated against the adversarial trap suite, and the Python backend was written with AI assistance and verified by unit tests, coverage measurement, and the trap scenarios. This manuscript was drafted with AI assistance and reviewed, edited, and validated by the human author, who made the core design decisions and verified all technical claims against the repository.

Acknowledgements

We thank the open-source communities of the Fable Method (Sahir619, 2026) and of the Karpathy Development Cycle for the methodologies that inspired this work, and the maintainers of the related tools surveyed in the State of the field section for the comparison that sharpened Kata's design.

References

- Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., & Zhou, D. (2024). Large language models cannot self-correct reasoning yet. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2310.01798>
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can language models resolve real-world GitHub issues? *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2310.06770>

- LerianStudio. (2026). Ring: Skills library for AI agents. In *GitHub repository*. GitHub. <https://github.com/LerianStudio/ring>
- Sahir619. (2026). The fable method: The fable workflow. In *GitHub repository*. GitHub. <https://github.com/Sahir619/fable-method>
- TeamSPWK. (2026). Nova: AI agent ops for claude code. In *GitHub repository*. GitHub. <https://github.com/TeamSPWK/nova>
- vlad-ko. (2026). Claude-wizard: Turn claude code from a fast coder into a senior software architect. In *GitHub repository*. GitHub. <https://github.com/vlad-ko/claude-wizard>
- Zhao, B., Srikanth, D., Wu, Y., & Jiang, Z. (2026). SpecBench: Measuring reward hacking in long-horizon coding agents. *arXiv Preprint arXiv:2605.21384*. <https://arxiv.org/abs/2605.21384>